TASK LEVEL - BEGINNER

SNEHAL SANTOSH JADHAV - IICT MGMU

Python Visualization Libraries :- Matplotlib & Seaborn

INTRODUCTION

Data visualization is the process of representing data in graphical or pictorial format. It helps in understanding trends, patterns, and insights from data. Python provides powerful libraries for data visualization. In this guide, we cover two of the most widely used libraries — Matplotlib and Seaborn.

Part 1 :- Matplotlib

Matplotlib is a foundational Python library used for creating static, animated, and interactive visualizations. It gives complete control over every element of a plot such as axes, labels, colors, and layout. It is widely used for publication-quality charts.

Key Features:

- Highly customizable
- Works with NumPy arrays
- Supports subplots and multi-panel figures
- Object-oriented API

1. Line Chart

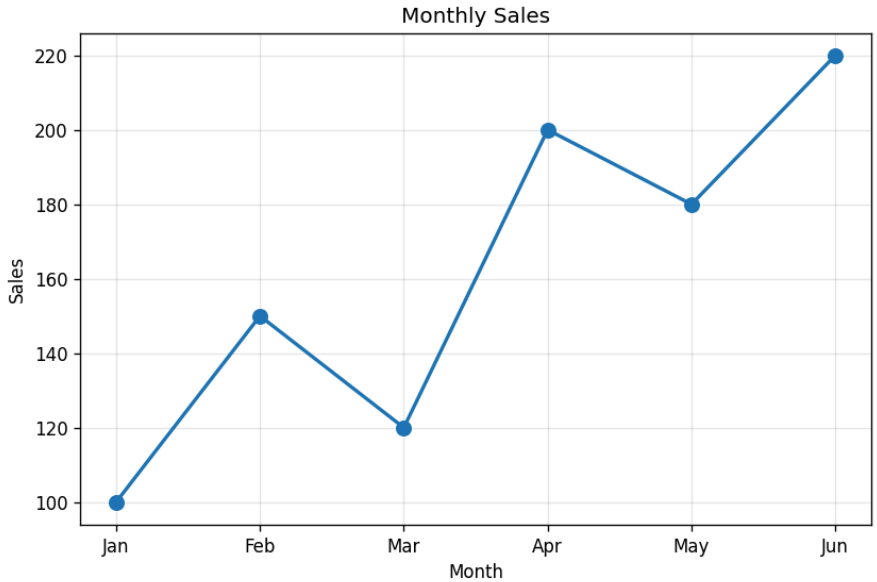
A line chart displays data points connected by straight lines. It is best used to show trends over time or continuous data such as monthly sales, temperature changes, or stock prices.

When to use: When you want to show how a value changes over a period of time.

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [100, 150, 120, 200, 180, 220]

plt.figure(figsize=(8, 5))
plt.plot(months, sales, marker='o', linewidth=2, markersize=8)
plt.title('Monthly Sales')
plt.xlabel('Month')
```

```
plt.ylabel('Sales')
plt.grid(True, alpha=0.3)
plt.show()
```

O/P:



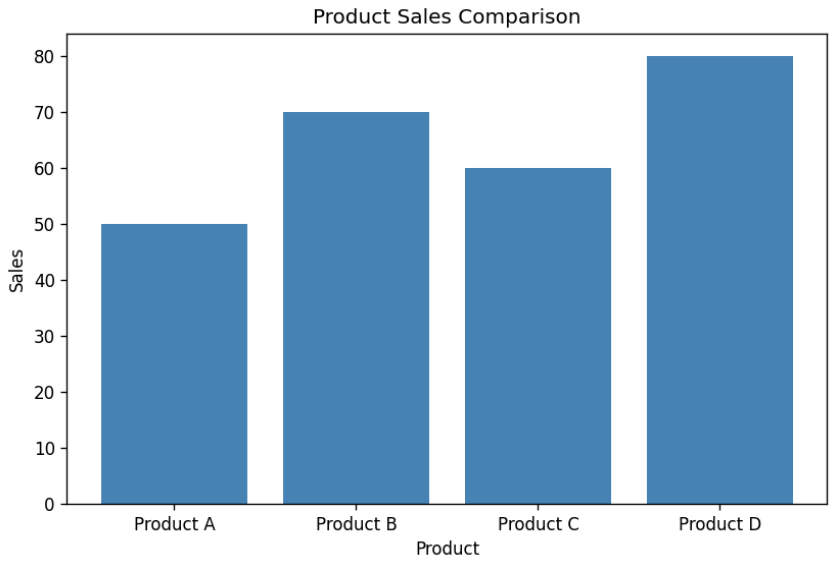
2. Bar Chart

A bar chart represents categorical data using rectangular bars. The height of each bar is proportional to the value it represents. It is used to compare different categories side by side.

When to use: When comparing quantities across different categories like products, regions, or groups.

```
import matplotlib.pyplot as plt



categories = ['Product A', 'Product B', 'Product C', 'Product D']

values = [50, 70, 60, 80]



plt.figure(figsize=(8, 5))

plt.bar(categories, values, color='steelblue')

plt.title('Product Sales Comparison')

plt.xlabel('Product')
```

```
plt.ylabel('Sales')

plt.show()
```

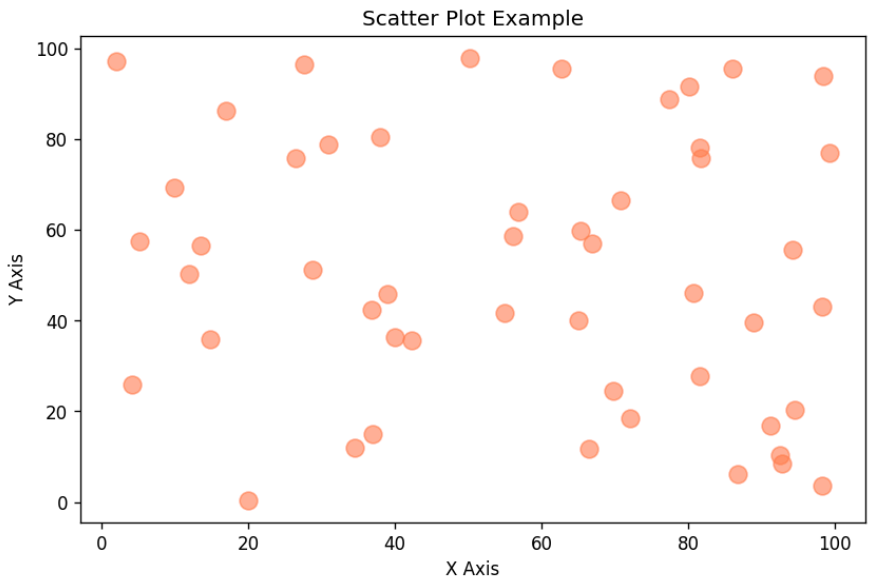


3. Scatter Plot

A scatter plot displays values for two variables as points on a graph. It is used to identify relationships, correlations, or clusters between two continuous variables.

When to use: When you want to see if there is a relationship or pattern between two variables.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.random.rand(50) * 100
y = np.random.rand(50) * 100

plt.figure(figsize=(8, 5))
plt.scatter(x, y, alpha=0.6, s=100, color='coral')
plt.title('Scatter Plot Example')
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.show()
```

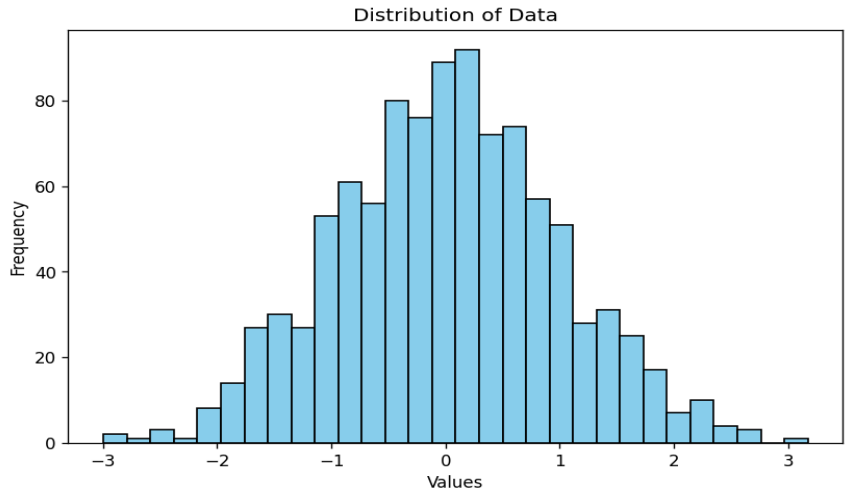


4. Histogram

A histogram shows the frequency distribution of a single variable. The data is divided into bins and the height of each bar shows how many values fall in that range.

When to use: When you want to understand how your data is distributed whether it is normal, skewed, or spread out.

```
import matplotlib.pyplot as plt
import numpy as np

data = np.random.randn(1000)

plt.figure(figsize=(8, 5))
plt.hist(data, bins=30, color='skyblue', edgecolor='black')
plt.title('Distribution of Data')
plt.xlabel('Values')
plt.ylabel('Frequency')
plt.show()
```

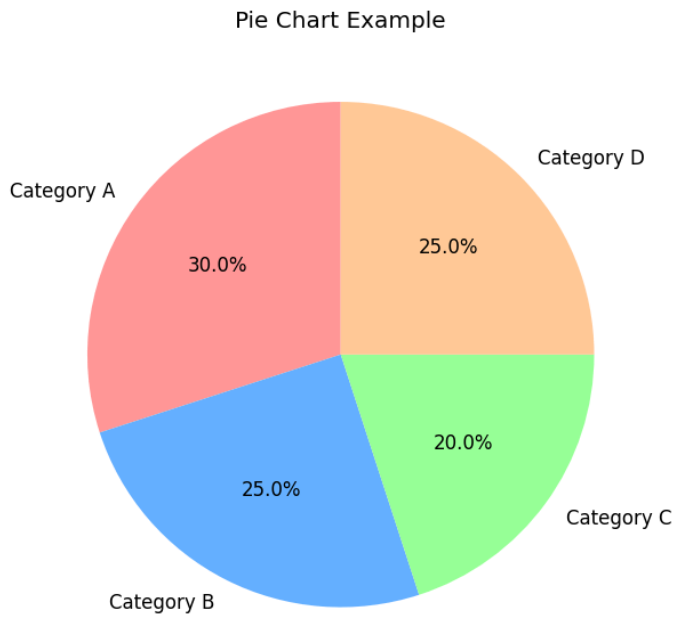


5. Pie Chart

A pie chart is a circular chart divided into slices to show part to whole relationships. Each slice represents a proportion of the total.

When to use: When you want to show percentage or proportional breakdown of categories.

```
import matplotlib.pyplot as plt



labels = ['Category A', 'Category B', 'Category C', 'Category D']

sizes = [30, 25, 20, 25]

colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99']



plt.figure(figsize=(8, 6))

plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%',
startangle=90)

plt.title('Pie Chart Example')

plt.show()
```



Part 2 :- Seaborn

Seaborn is a Python data visualization library built on top of Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics. It works very well with Pandas DataFrames and requires less code compared to Matplotlib.

Key Features:

- Built-in themes and beautiful default styles
- Deep integration with Pandas
- Ideal for statistical visualizations
- Less code needed for complex charts

7. Heatmap

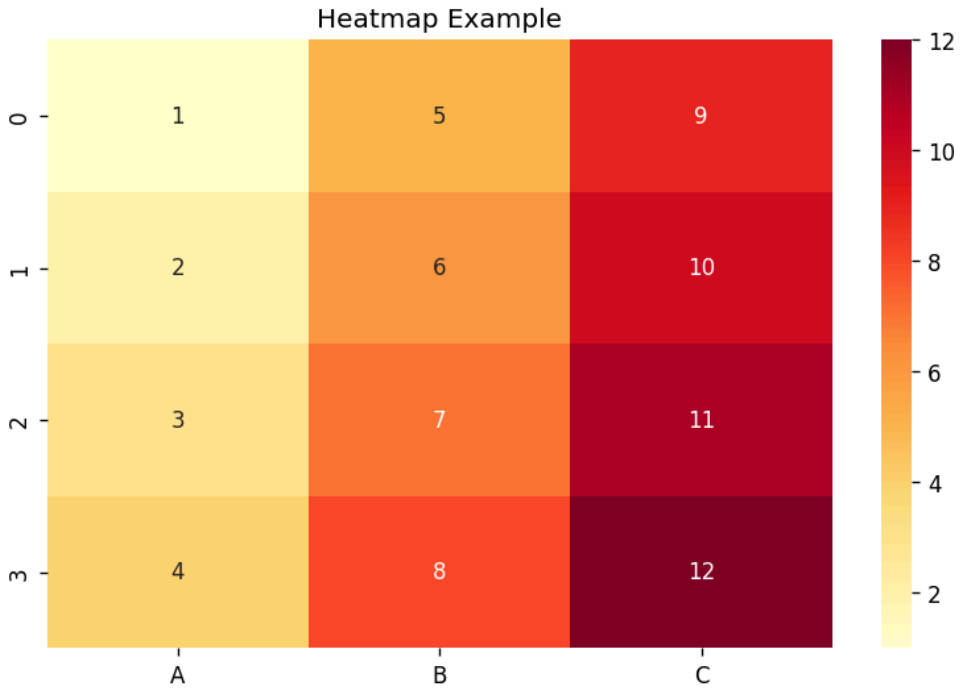
A heatmap displays data in a matrix format where individual values are represented as colors. It is commonly used to show correlation between variables or intensity of values across a grid.

When to use: When you want to visualize relationships between multiple variables at once or show a correlation matrix.

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

data = pd.DataFrame({
```

```
    'A': [1, 2, 3, 4],
    'B': [5, 6, 7, 8],
    'C': [9, 10, 11, 12]
})

plt.figure(figsize=(8, 5))
sns.heatmap(data, annot=True, cmap='YlOrRd', fmt='d')
plt.title('Heatmap Example')
plt.show()
```

O/P



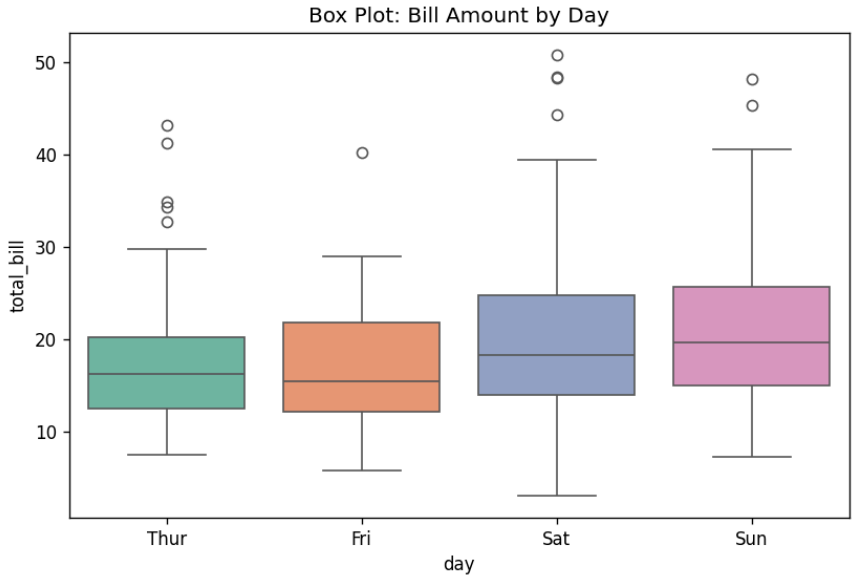
8. Box Plot

A box plot summarizes the distribution of data using five key statistics — minimum, first quartile, median, third quartile, and maximum. It also shows outliers as individual points.

When to use: When comparing distributions across different groups and spotting outliers.

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

plt.figure(figsize=(8, 5))
```

```
sns.boxplot(x='day', y='total_bill', data=tips, palette='Set2')
plt.title('Box Plot: Bill Amount by Day')
plt.show()
```

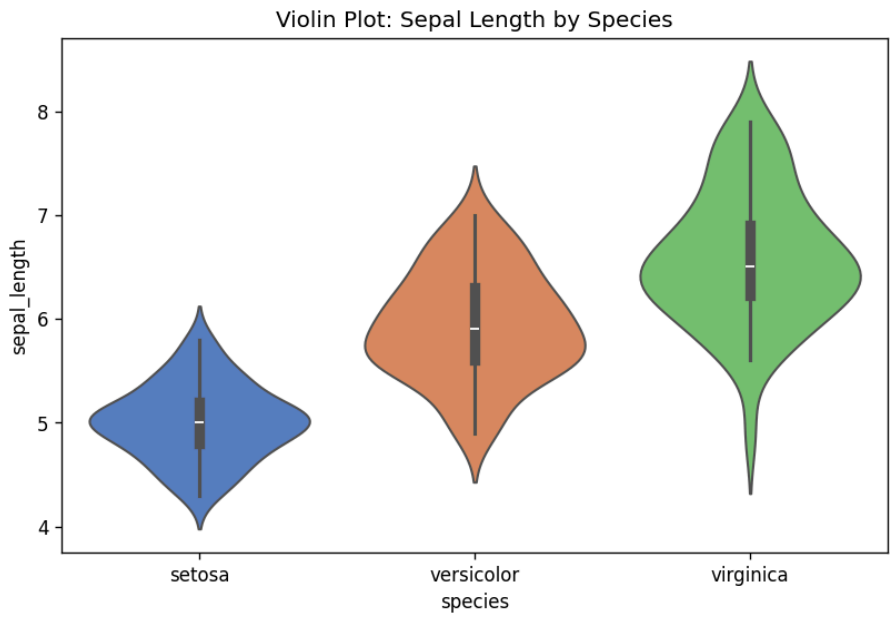


9. Violin Plot

A violin plot combines a box plot and a KDE plot. It shows the full distribution shape of the data along with summary statistics. It gives more detail than a box plot.

When to use: When you want to see the full shape of the distribution across categories, not just the summary.

```
import seaborn as sns
import matplotlib.pyplot as plt

iris = sns.load_dataset('iris')

plt.figure(figsize=(8, 5))
sns.violinplot(x='species', y='sepal_length', data=iris,
palette='muted')
plt.title('Violin Plot: Sepal Length by Species')
plt.show()
```

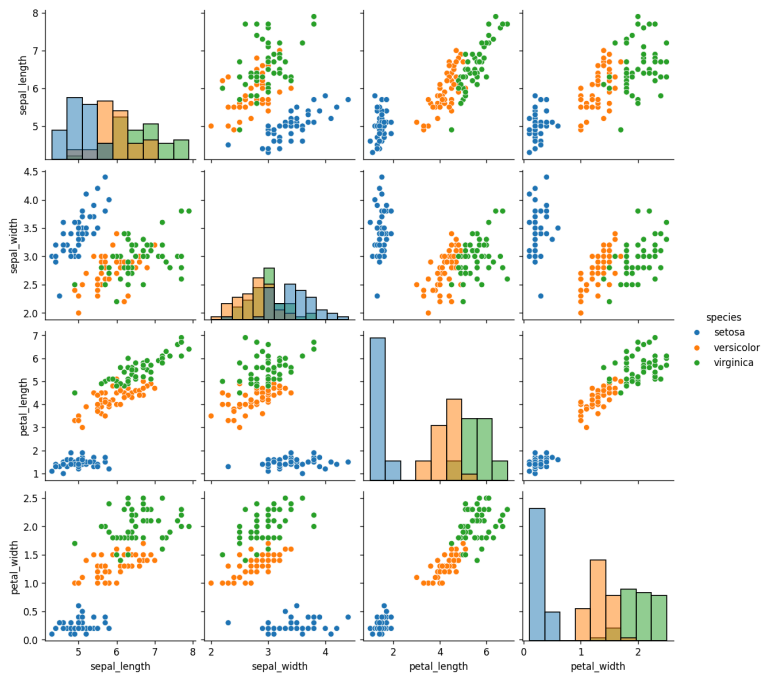


10. Pair Plot

A pair plot creates a grid of scatter plots for every pair of variables in a dataset. The diagonal shows the distribution of each individual variable. It is very useful for exploratory data analysis.

When to use: When you want to explore relationships between all features in a dataset at once.

```
import seaborn as sns

iris = sns.load_dataset('iris')
sns.pairplot(iris, hue='species', diag_kind='hist')
plt.show()
```

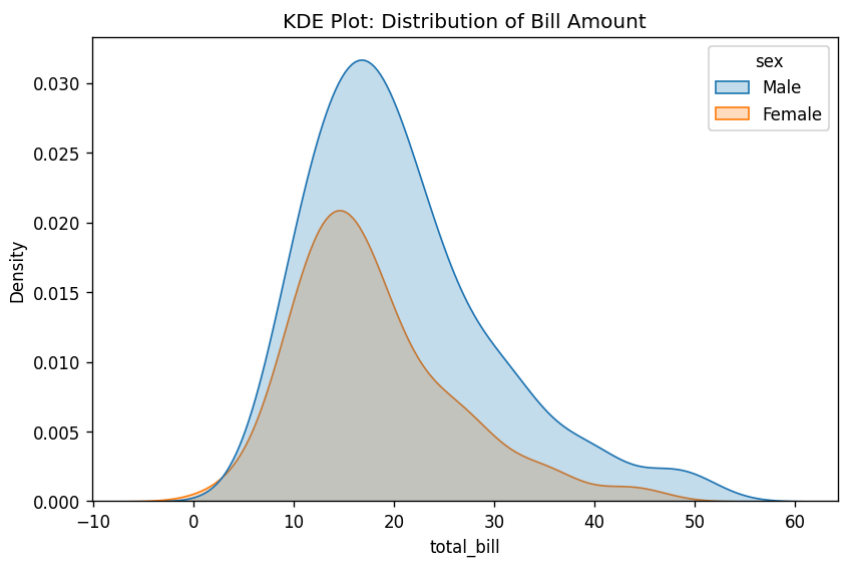


11. KDE Plot

KDE stands for Kernel Density Estimate. A KDE plot shows the smooth probability density of a variable. It is like a smoothed version of a histogram.

When to use: When you want to visualize and compare the distribution of one or more groups in a smooth, continuous form.

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

plt.figure(figsize=(8, 5))
sns.kdeplot(data=tips, x='total_bill', hue='sex', fill=True)
plt.title('KDE Plot: Distribution of Bill Amount')
plt.show()
```

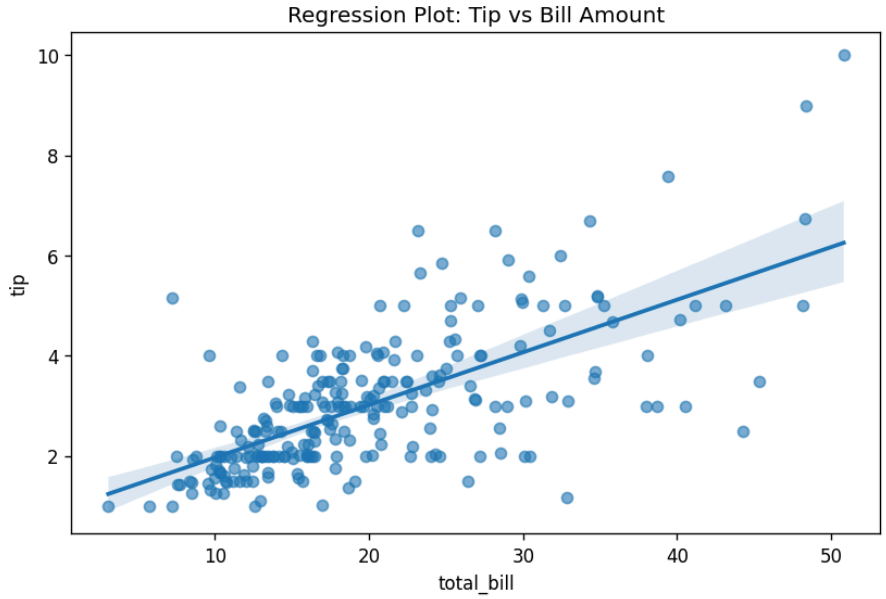


12. Regression Plot

A regression plot shows a scatter plot along with a fitted regression line. It also displays the confidence interval around the line, showing how reliable the trend is.

When to use: When you want to visualize the linear relationship between two variables and see the trend line.

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')

plt.figure(figsize=(8, 5))
sns.regplot(x='total_bill', y='tip', data=tips,
scatter_kws={'alpha':0.6})
plt.title('Regression Plot: Tip vs Bill Amount')
plt.show()
```



Conclusion

Working with Matplotlib and Seaborn has been a really interesting experience. These two libraries together cover almost everything you need when it comes to data visualization in Python.

Matplotlib gives you full control over your charts, you can customize every little detail exactly the way you want. Seaborn, on the other hand, makes things much easier and faster, especially when you're working with real datasets and want good-looking charts without writing too much code.

What I realized through this project is that visualization is not just about making pretty graphs, it's about telling a story with your data. The right chart can instantly show patterns and insights that would take pages of text to explain otherwise.